254383 Algorithm Design and Analysis

อ.พิเศษพงศ์ สุธาพันธ์
ภาควิชาวิทยาการคอมพิวเตอร์และเทคโนโลยีสารสนเทศ

Week 5

Divide-and-conquer

อ.พิเศษพงศ์ สุธาพันธ์
ภาควิชาวิทยาการคอมพิวเตอร์และเทคโนโลยีสารสนเทศ

Divide-and-conquer

วัตถุประสงค์

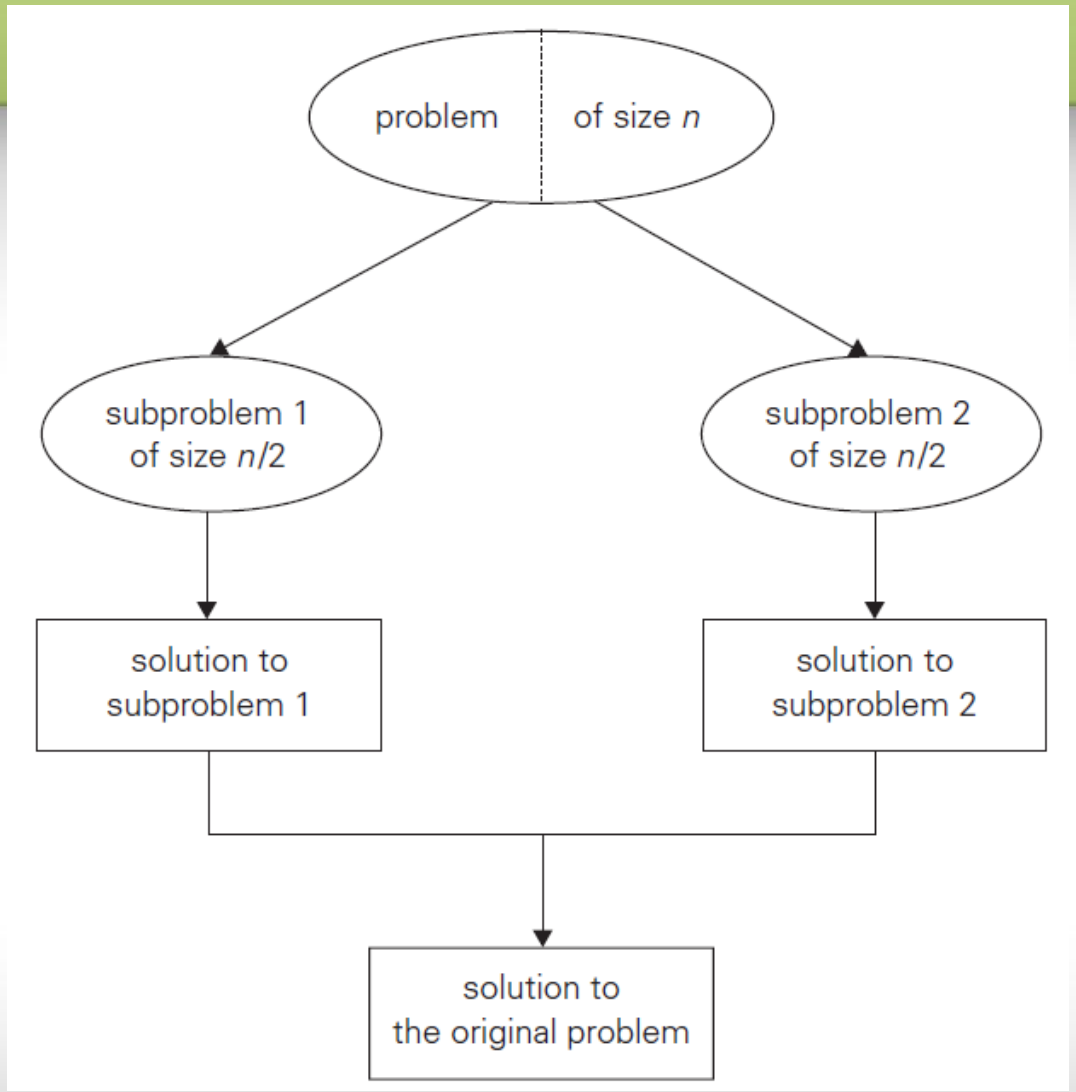
1. รู้ถึงการทำงานของ เทคนิค Divide-and-conquer
2. นำไปประยุกต์ใช้งานได้

Divide-and-conquer

มีแนวทางดำเนินงานต่อไปนี้

- ปัญหาจะถูกแบ่งออกเป็นปัญหาย่อย ๆชนิดเดียวกันหลายส่วน
- แก้ไขปัญหาย่อย
- คำตอบของปัญหาย่อยจะถูกนำมารวมกันเพื่อหาคำตอบปัญหาเดิม

Divide-and-conquer



Divide-and-conquer technique (typical case).

Divide-and-conquer

ปัญหามีข้อมูลขนาด n แบ่งข้อมูลเป็น b ชุด ชุดละ n/b และ a จำนวนครั้งในการแก้ปัญหา

$a \geq 1$ และ $b > 1$

Recurrence for running time T(n)

$$T(n) = aT(n/b) + f(n)$$

If $f(n) \in \Theta(n^d) where\ d \geq 0$

Divide-and-conquer

Efficiency analysis of divide-and conquer algorithm

Master Theorem If $f(n) \in \Theta(n^d) where\ d \geq 0$

$$T(n) \in \begin{cases} \Theta(n^d) & if\ a < b^d \\ \Theta(n^d \log n) & if\ a = b^d \\ \Theta(n^{\log_b a}) & if\ a > b^d \end{cases}$$

Divide-and-conquer

Efficiency analysis of divide-and conquer algorithm

สมมุติแบ่งปัญหาออกเป็น 2 ส่วน

เวลาที่ใช้

$$T(n) = \begin{cases} 0 & n = 1 \\ T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1 & n > 1 \end{cases}$$

$$T(n) = \begin{cases} 0 & n = 1 \\ 2T(n/2) + 1 & n > 1 \end{cases}$$

Divide-and-conquer

Efficiency analysis of divide-and conquer algorithm

ให้ n = 2^m โดยใช้ backward substitution

$$
\begin{array}{lcll}
T(2^m) & = & 2T(2^{m-1}) + 1 & \\
T(2^{m-1}) & = & 2T(2m-2) + 1 & * 2 \\
 & & & \\
T(2) & = & 2T(1) + 1 & * 2^{m-1} \\
T(1) & = & 0 &
\end{array}
$$

Divide-and-conquer

Efficiency analysis of divide-and conquer algorithm

ให้ n = 2^m โดยใช้ backward substitution

$$T(n) = 1 + 2^1 + 2^{m-1}$$

$$= 2^m - 1 = n - 1$$

$$T(n) = n - 1 \qquad \Theta(n)$$

Divide-and-conquer

Efficiency analysis of divide-and conquer algorithm

ต.ย. Input of size n = 2^k

$$A(n) = 2A\left(\frac{n}{2}\right) + 1$$

n^d

ให้ a = 2 , b = 2 และ d = 0

$$a > b^d$$

$$A(n) \in \Theta\left(n^{\log_b a}\right) = \Theta\left(n^{\log_2 2}\right) = \Theta(n)$$

Divide-and-conquer

5.1 Mergesort

http://10.31.26.26/~phisetphong/algorithm/mergesort.html

Mergesort

ALGORITHM *Mergesort*$(A[0..n-1])$

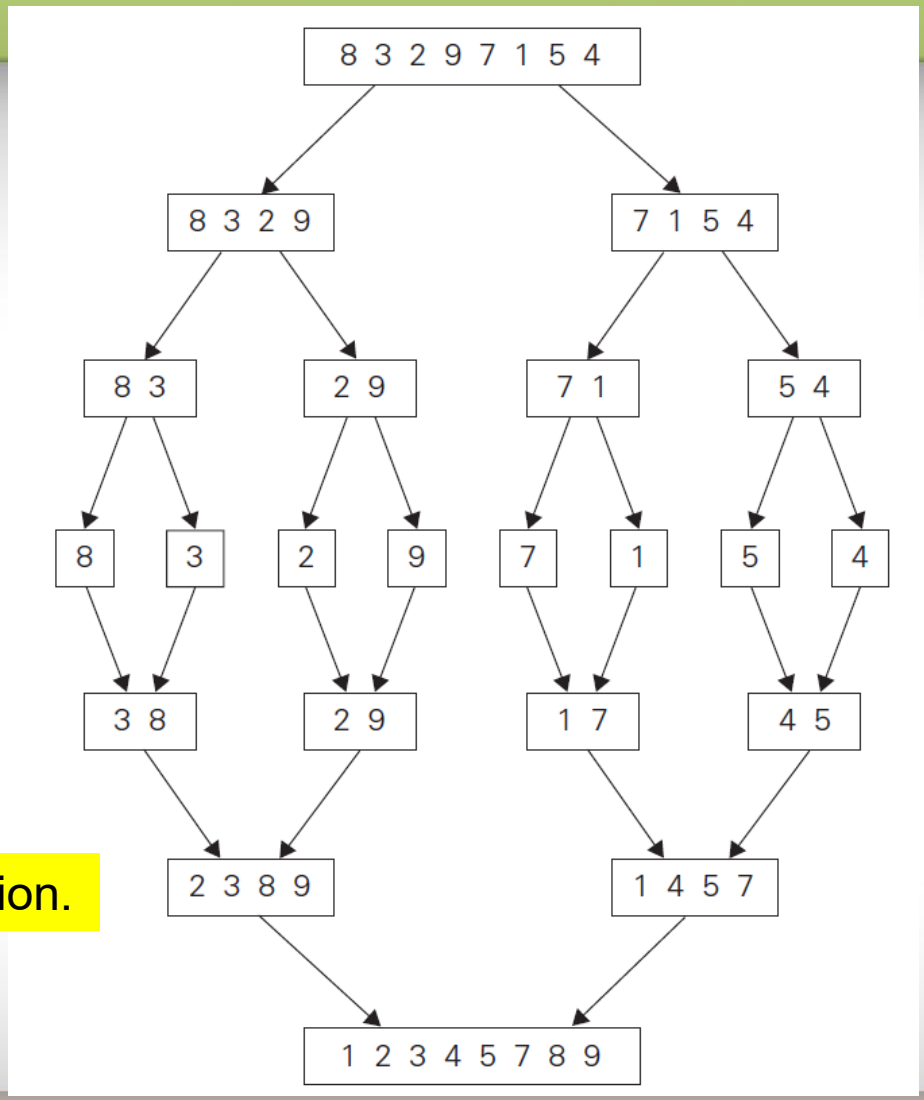
//Sorts array $A[0..n-1]$ by recursive mergesort
//Input: An array $A[0..n-1]$ of orderable elements
//Output: Array $A[0..n-1]$ sorted in nondecreasing order
if $n > 1$
    copy $A[0..\lfloor n/2 \rfloor - 1]$ to $B[0..\lfloor n/2 \rfloor - 1]$
    copy $A[\lfloor n/2 \rfloor..n-1]$ to $C[0..\lceil n/2 \rceil - 1]$
    Mergesort$(B[0..\lfloor n/2 \rfloor - 1])$
    Mergesort$(C[0..\lceil n/2 \rceil - 1])$
    Merge(B, C, A) //see below

Mergesort

ALGORITHM *Merge*$(B[0..p-1], C[0..q-1], A[0..p+q-1])$

//Merges two sorted arrays into one sorted array
//Input: Arrays $B[0..p-1]$ and $C[0..q-1]$ both sorted
//Output: Sorted array $A[0..p+q-1]$ of the elements of B and C
$i \leftarrow 0;\ j \leftarrow 0;\ k \leftarrow 0$
while $i < p$ **and** $j < q$ **do**
    if $B[i] \leq C[j]$
        $A[k] \leftarrow B[i];\ i \leftarrow i+1$
    else $A[k] \leftarrow C[j];\ j \leftarrow j+1$
    $k \leftarrow k+1$
if $i = p$
    copy $C[j..q-1]$ to $A[k..p+q-1]$
else copy $B[i..p-1]$ to $A[k..p+q-1]$

Mergesort



Example of mergesort operation.

Mergesort

Efficient of mergesort

$$C(n) = 2C\left(\frac{n}{2}\right) + C_{merge}(n)\ ;\ for\ n > 1, C(1) = 0$$

Worst case

$C_{merge}(n) = n - 1$; In worst case

$$C_{worst}(n) = 2C_{worst}(n/2) + n - 1\ ;$$

for n > 1, $C_{worst}(1) = 0$

$$C_{worst}(n) = \Theta(n\ log\ n)$$

Mergesort

Efficient of mergesort

$C_{merge}(n) \rightarrow \mathbf{f(n)}$ เป็นสมาชิกของ $\Theta(n)$

Worst case

$C_{merge}(n) = n - 1$; In worst case

$C_{worst}(n) = 2C_{worst}(n/2) + n - 1$;

for n > 1, $C_{worst}(1) = 0$

$C_{worst}(n) = \Theta(n \log n)$

Divide-and-conquer

5.2 Quicksort

http://10.31.26.26/~phisetphong/algorithm/quicksort.html

Quicksort

รูปแบบ

$$\underbrace{A[0] \ldots A[s-1]}_{\text{all are} \leq A[s]} \quad A[s] \quad \underbrace{A[s+1] \ldots A[n-1]}_{\text{all are} \geq A[s]}$$

Quicksort

ALGORITHM *Quicksort*$(A[l..r])$

//Sorts a subarray by quicksort
//Input: Subarray of array $A[0..n-1]$, defined by its left and right
// indices l and r
//Output: Subarray $A[l..r]$ sorted in nondecreasing order
if $l < r$
  $s \leftarrow Partition(A[l..r])$ //s is a split position
  $Quicksort(A[l..s-1])$
  $Quicksort(A[s+1..r])$

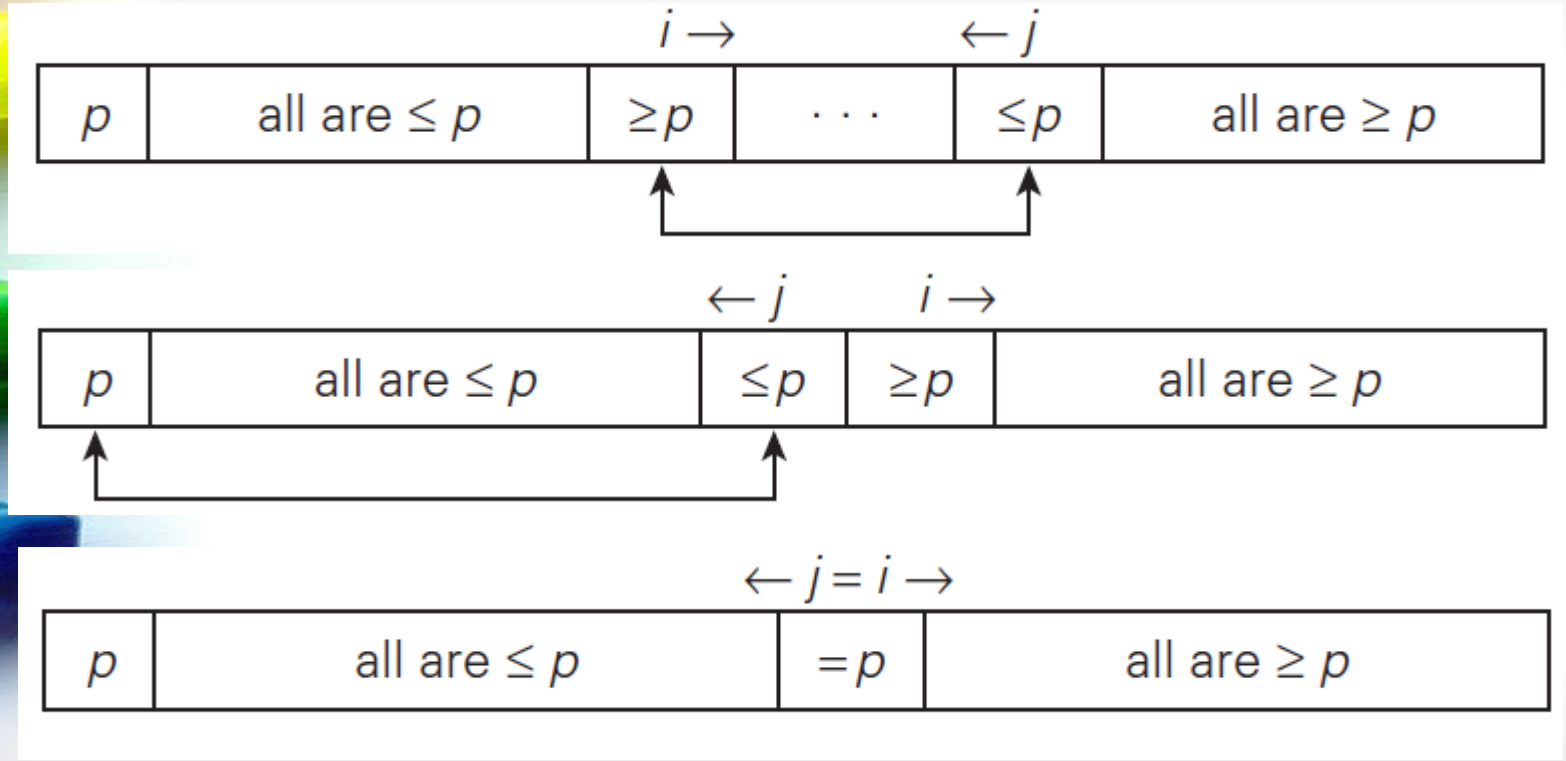
Quicksort

ALGORITHM *HoarePartition*$(A[l..r])$

```
//Partitions a subarray by Hoare's algorithm, using the first element
//          as a pivot
//Input: Subarray of array A[0..n − 1], defined by its left and right
//          indices l and r (l < r)
//Output: Partition of A[l..r], with the split position returned as
//          this function's value
p ← A[l]
i ← l; j ← r + 1
repeat
    repeat i ← i + 1 until A[i] ≥ p
    repeat j ← j − 1 until A[j] ≤ p
    swap(A[i], A[j])
until i ≥ j
swap(A[i], A[j])    //undo last swap when i ≥ j
swap(A[l], A[j])
return j
```

Quicksort

รูปแบบการทำงาน



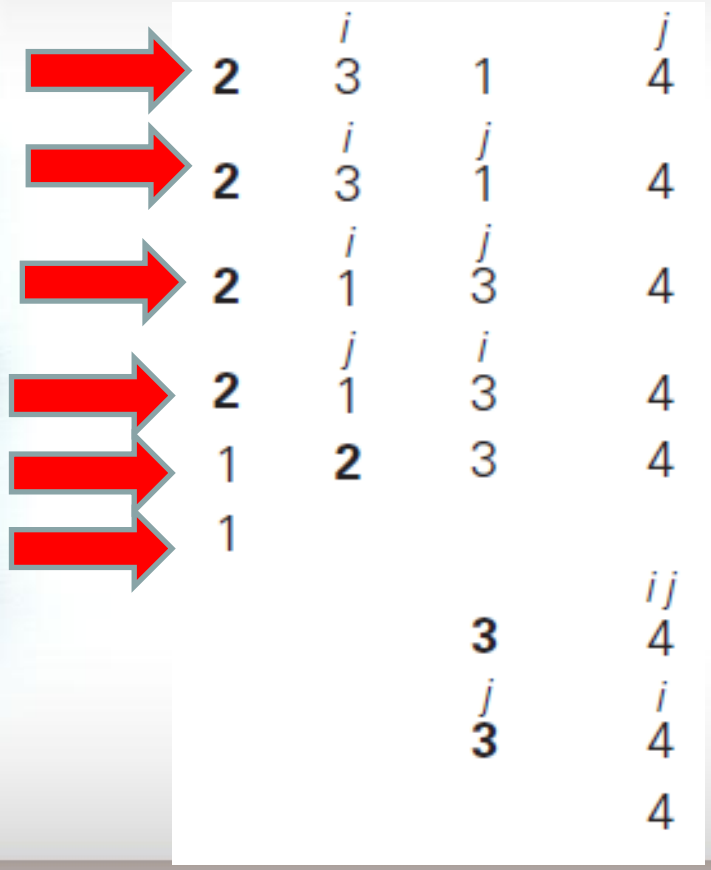
Quicksort

การทำงาน

0	1	2	3	4	5	6	7
	i						*j*
5	3	1	9	8	2	4	7
			i			*j*	
5	3	1	9	8	2	4	7
			i			*j*	
5	3	1	4	8	2	9	7
				i	*j*		
5	3	1	4	8	2	9	7
				i	*j*		
5	3	1	4	2	8	9	7
				j	*i*		
5	3	1	4	2	8	9	7
2	3	1	4	**5**	8	9	7

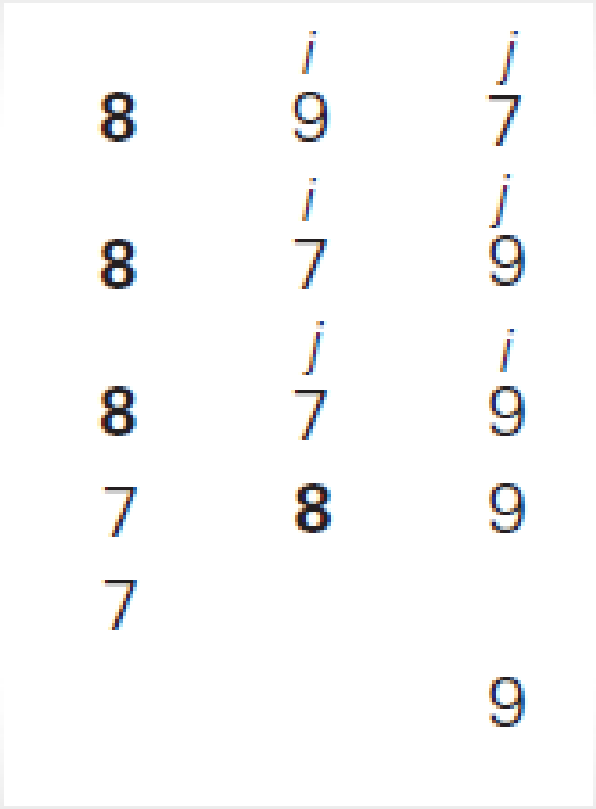
Quicksort

การทำงาน



Quicksort

การทำงาน



Quicksort

worst case ในกรณีที่ข้อมูลถูกจัดเรียงมาแล้ว

$$C_{worst}(n) = (n+1) + n + \cdots + 3 = \frac{(n+1)(n+2)}{2} - 3 \in \Theta(n^2).$$

average case

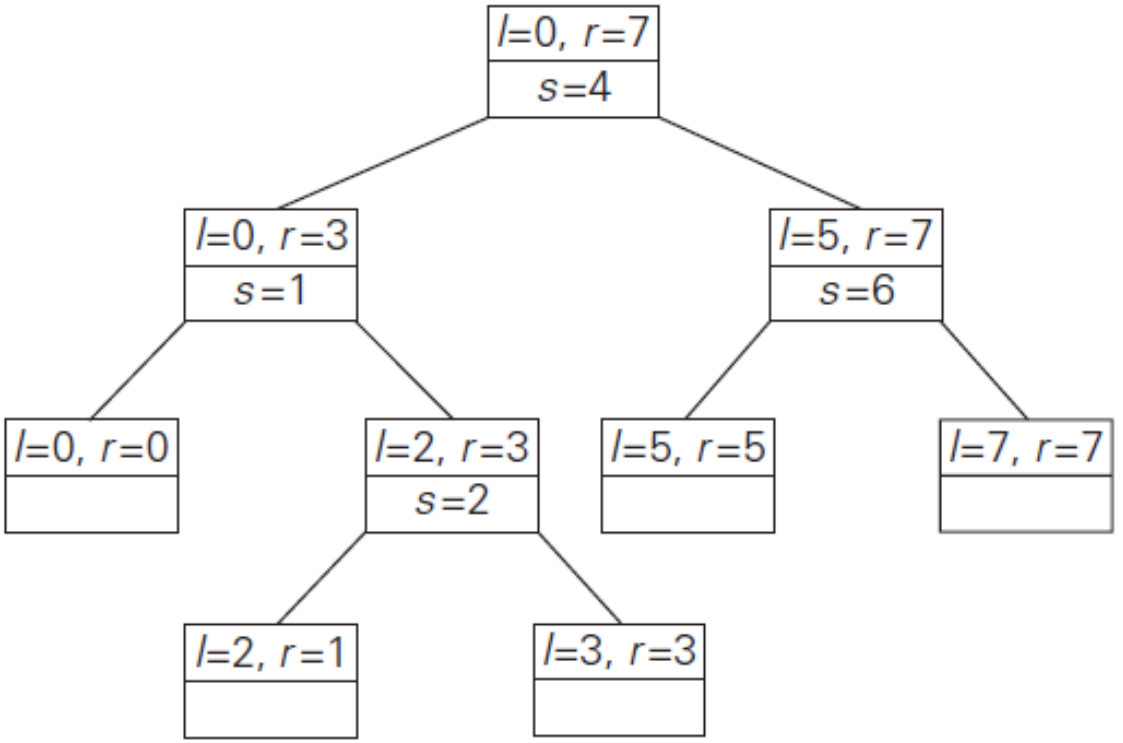
$$C_{avg}(n) = \frac{1}{n}\sum_{s=0}^{n-1}[(n+1) + C_{avg}(s) + C_{avg}(n-1-s)] \quad \text{for } n > 1,$$

$$C_{avg}(0) = 0, \quad C_{avg}(1) = 0.$$

$$C_{avg}(n) \approx 2n \ln n \approx 1.39n \log_2 n.$$

Quicksort

การทำงาน



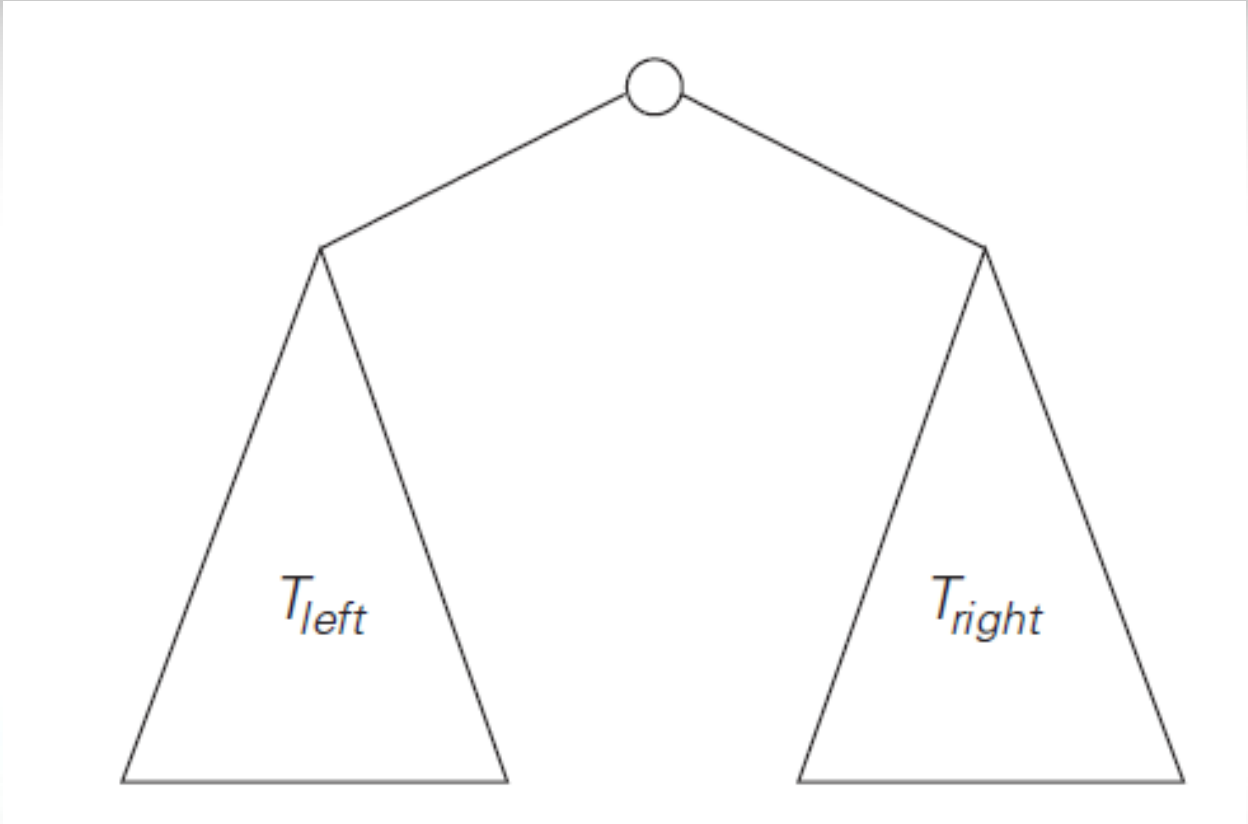
Tree of recursive calls to Quicksort with input values l and r of subarray bounds and split position s of a partition obtained

Divide-and-conquer

5.3 Binary Tree Traversals and Related Properties

http://10.31.26.26/~phisetphong/algorithm/binarySearchTree.html

Binary Tree Traversals and Related Properties



Standard representation of a binary tree.

Binary Tree Traversals and Related Properties

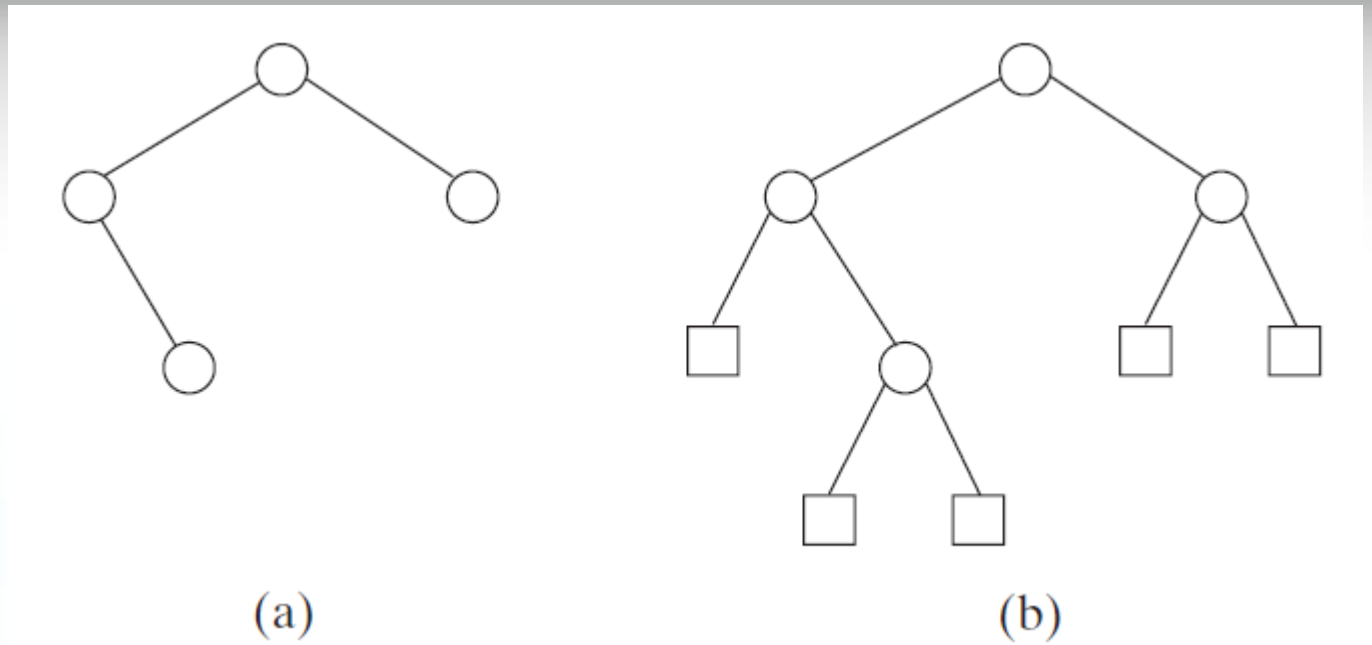
ALGORITHM *Height*(T)

//Computes recursively the height of a binary tree
//Input: A binary tree T
//Output: The height of T
if $T = \varnothing$ **return** -1
else return $\max\{Height(T_{left}), Height(T_{right})\} + 1$

recurrence relation for A(n(T)):

$$A(n(T)) = A(n(T_{left})) + A(n(T_{right})) + 1 \quad \text{for } n(T) > 0,$$
$$A(0) = 0.$$

Binary Tree Traversals and Related Properties



Binary tree (on the left) and its extension (on the right). Internal nodes are shown as circles; external nodes are shown as squares.

Binary Tree Traversals and Related Properties

ให้ x เป็น จำนวน external nodes

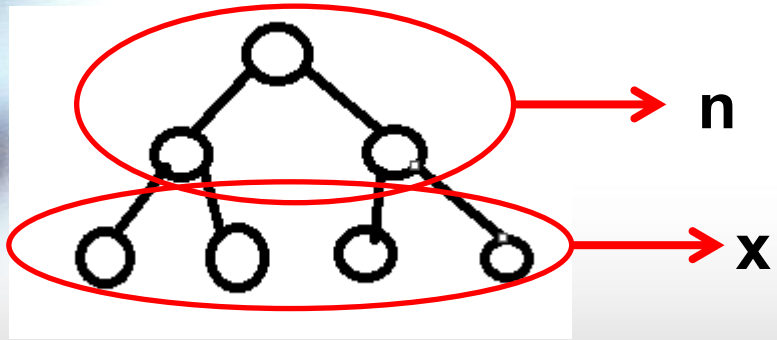
ให้ n เป็น จำนวน internal nodes

Full binary tree

$x = n + 1$

$2n + 1 = x + n$

จำนวนโหนดทั้งหมด



Binary Tree Traversals and Related Properties

Full binary tree

จำนวนการเปรียบเทียบ

$$C(n) = n + x = 2n + 1$$

จำนวนการบวก

$$A(n) = n$$

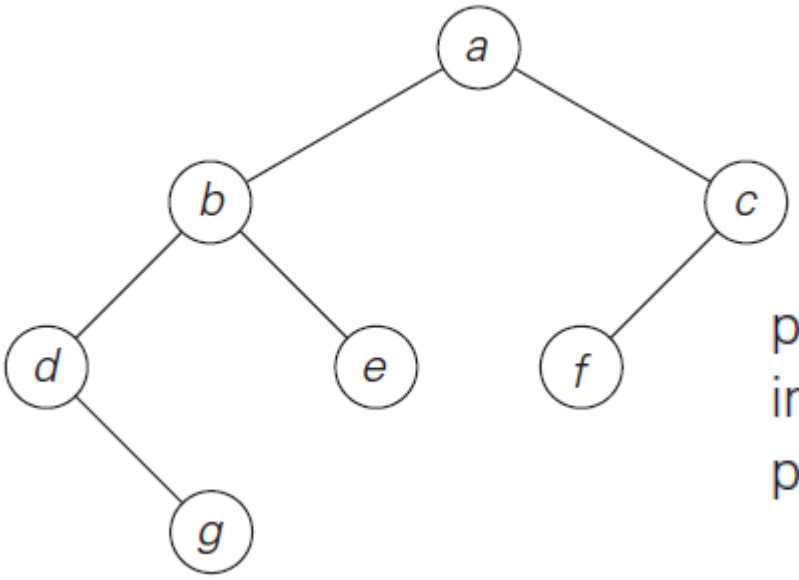
Binary Tree Traversals and Related Properties

การท่องใน binary tree มี 3 แบบ

1. Preorder traversal เยี่ยม root แล้วไปต้นไม้ย่อยซ้ายและขวา
2. Inorder traversal เยี่ยม root หลังต้นไม้ย่อยซ้ายและก่อนต้นไม้ย่อยขวา
3. Postorder traversal เยี่ยม root หลังต้นไม้ย่อยซ้ายและขวา

Binary Tree Traversals and Related Properties

ต.ย.



preorder: *a, b, d, g, e, c, f*
inorder: *d, g, b, e, a, f, c*
postorder: *g, d, e, b, f, c, a*

Binary tree and its traversals.

Divide-and-conquer

5.4 Multiplication of Large Integers and Strassen's Matrix Multiplication

Multiplication of Large Integers

$$23 = 2 \cdot 10^1 + 3 \cdot 10^0 \quad \text{and} \quad 14 = 1 \cdot 10^1 + 4 \cdot 10^0.$$

$$\begin{aligned} 23 * 14 &= (2 \cdot 10^1 + 3 \cdot 10^0) * (1 \cdot 10^1 + 4 \cdot 10^0) \\ &= (2 * 1)10^2 + (2 * 4 + 3 * 1)10^1 + (3 * 4)10^0. \end{aligned}$$

$$2 * 4 + 3 * 1 = (2 + 3) * (1 + 4) - 2 * 1 - 3 * 4.$$

Multiplication of Large Integers

สูตร การคูณเลข 2 หลัก สองจำนวน

$$a = a_1 a_0 \text{ and } b = b_1 b_0$$

$$c = a * b = c_2 10^2 + c_1 10^1 + c_0$$

where

$c_2 = a_1 * b_1$ is the product of their first digits,

$c_0 = a_0 * b_0$ is the product of their second digits,

$c_1 = (a_1 + a_0) * (b_1 + b_0) - (c_2 + c_0)$ is the product of the sum of the a's digits and the sum of the b's digits minus the sum of c_2 and c_0.

Multiplication of Large Integers

สูตร การคูณเลข 2 หลัก สองจำนวน (เพิ่มเติม)

$$a = a_1 10^{n/2} + a_0 \text{ and } b = b_1 10^{n/2} + b_0$$

$$\begin{aligned} c = a * b &= (a_1 10^{n/2} + a_0) * (b_1 10^{n/2} + b_0) \\ &= (a_1 * b_1)10^n + (a_1 * b_0 + a_0 * b_1)10^{n/2} + (a_0 * b_0) \\ &= c_2 10^n + c_1 10^{n/2} + c_0, \end{aligned}$$

where

$c_2 = a_1 * b_1$ is the product of their first halves,

$c_0 = a_0 * b_0$ is the product of their second halves,

$c_1 = (a_1 + a_0) * (b_1 + b_0) - (c_2 + c_0)$ is the product of the sum of the a's halves and the sum of the b's halves minus the sum of c_2 and c_0.

Multiplication of Large Integers

Recurrence ของการคูณ

$$M(n) = 3M(n/2) \quad \text{for } n > 1, \quad M(1) = 1$$

Solving it by backward substitutions for $n = 2^k$ yields

$$M(2^k) = 3M(2^{k-1}) = 3[3M(2^{k-2})] = 3^2 M(2^{k-2})$$
$$= \cdots = 3^i M(2^{k-i}) = \cdots = 3^k M(2^{k-k}) = 3^k$$

Since $k = \log_2 n$,

$$M(n) = 3^{\log_2 n} = n^{\log_2 3} \approx n^{1.585}.$$

Strassen's Matrix Multiplication

การคูณเมตริกซ์ 2*2 จำนวนสองเมตริกซ์เข้าด้วยกัน

$$\begin{bmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{bmatrix} = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix} * \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{bmatrix}$$

$$= \begin{bmatrix} m_1 + m_4 - m_5 + m_7 & m_3 + m_5 \\ m_2 + m_4 & m_1 + m_3 - m_2 + m_6 \end{bmatrix}$$

Strassen's Matrix Multiplication

การคูณเมตริกซ์ 2*2 จำนวนสองเมตริกซ์เข้าด้วยกัน

เมื่อ

$$m_1 = (a_{00} + a_{11}) * (b_{00} + b_{11}),$$
$$m_2 = (a_{10} + a_{11}) * b_{00},$$
$$m_3 = a_{00} * (b_{01} - b_{11}),$$
$$m_4 = a_{11} * (b_{10} - b_{00}),$$
$$m_5 = (a_{00} + a_{01}) * b_{11},$$
$$m_6 = (a_{10} - a_{00}) * (b_{00} + b_{01}),$$
$$m_7 = (a_{01} - a_{11}) * (b_{10} + b_{11}).$$

Strassen's Matrix Multiplication

ให้ A และ B มีขนาด n x n โดยที่ n คือ 2 ยกกำลังใด ๆ

สามารถแบ่ง A * B = C เป็น 4 ส่วน ดังรูป

$$\left[\begin{array}{c|c} C_{00} & C_{01} \\ \hline C_{10} & C_{11} \end{array}\right] = \left[\begin{array}{c|c} A_{00} & A_{01} \\ \hline A_{10} & A_{11} \end{array}\right] * \left[\begin{array}{c|c} B_{00} & B_{01} \\ \hline B_{10} & B_{11} \end{array}\right]$$

Strassen's Matrix Multiplication

$$C_{00} = A_{00} * B_{00} + A_{01} * B_{10}$$

หรือ

$$C_{00} = M_1 + M_4 - M_5 + M_7$$

Strassen's Matrix Multiplication

ถ้า M(n) เป็นจำนวนการคูณของ Streassen's Algorithm

Recurrence relation คือ

$$M(n) = 7M(n/2) \text{ for } n > 1 \text{ , } M(1) = 1$$

Strassen's Matrix Multiplication

เมื่อ $n = 2^k$

$$M(2^k) = 7M(2^{k-1}) = 7[7M(2^{k-2})] = 7^2 M(2^{k-2}) = \cdots$$
$$= 7^i M(2^{k-i}) \cdots = 7^k M(2^{k-k}) = 7^k.$$

Since $k = \log_2 n$,

$$M(n) = 7^{\log_2 n} = n^{\log_2 7} \approx n^{2.807},$$

Strassen's Matrix Multiplication

รวมทำการบวบลบ 18 ครั้ง

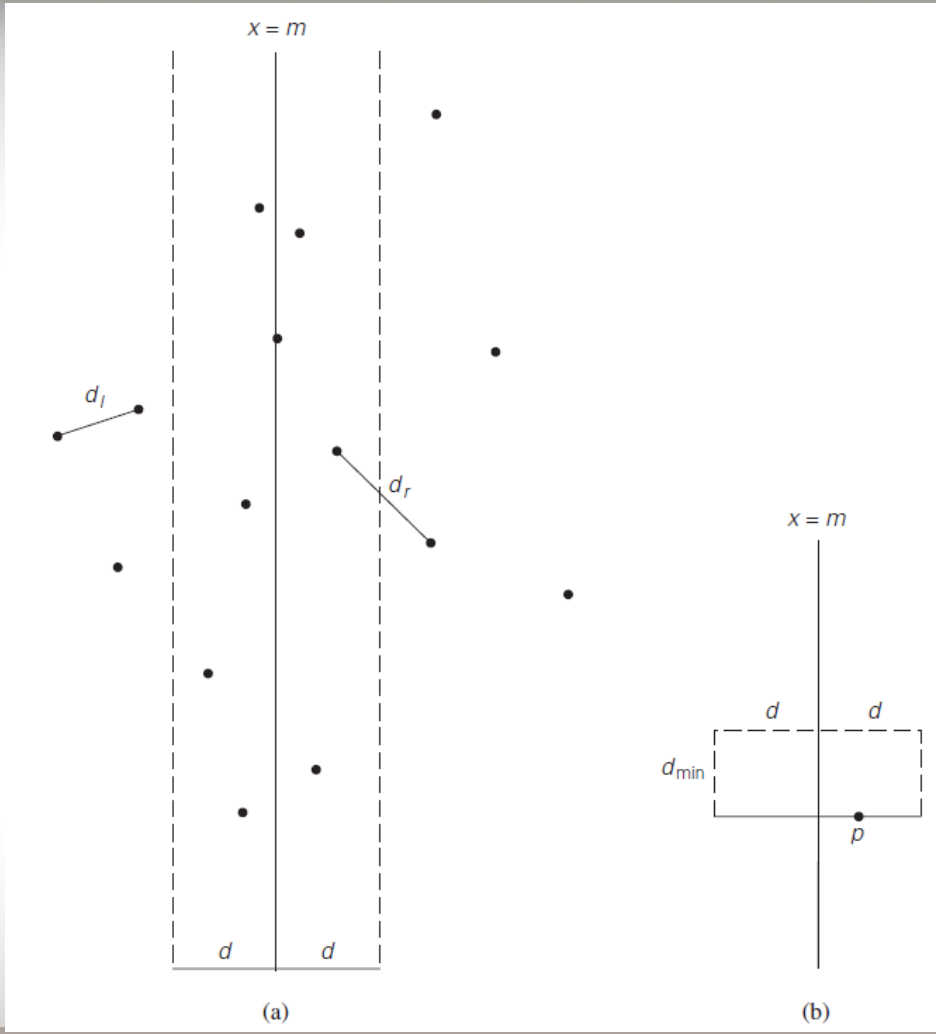
$$A(n) = 7A(n/2) + 18(n/2)^2 \quad \text{for } n > 1, \quad A(1) = 0.$$

$$A(n) \in \Theta(n^{\log_2 7})$$

Divide-and-conquer

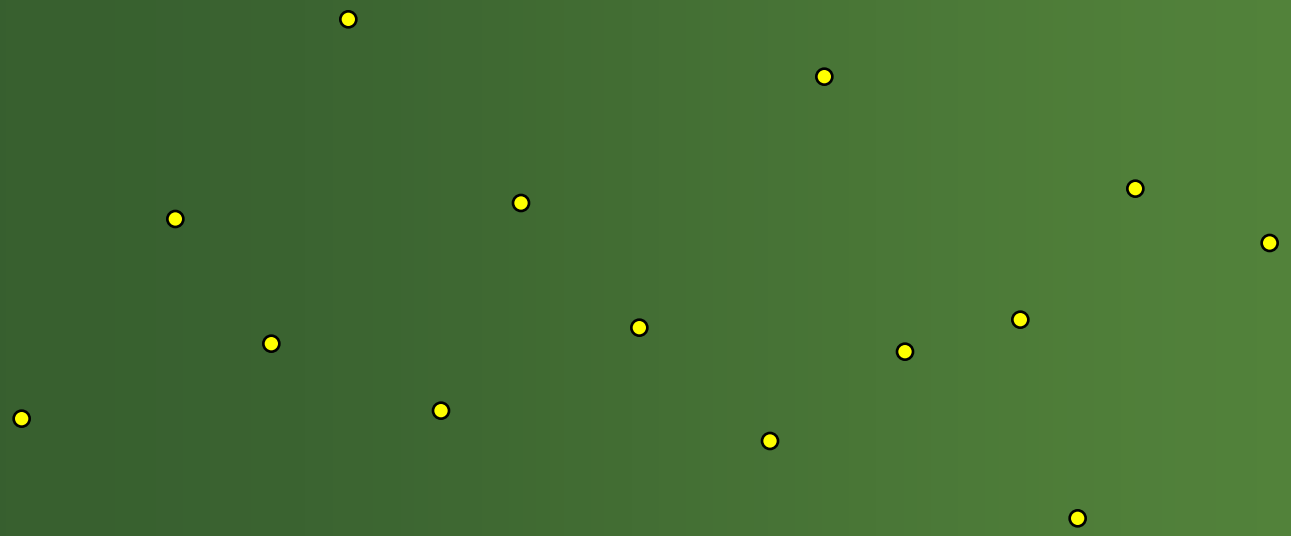
5.5 The Closest-Pair and Convex-Hull Problems by Divide-and Conquer

Closest-Pair Problem



Closest-Pair Algorithm

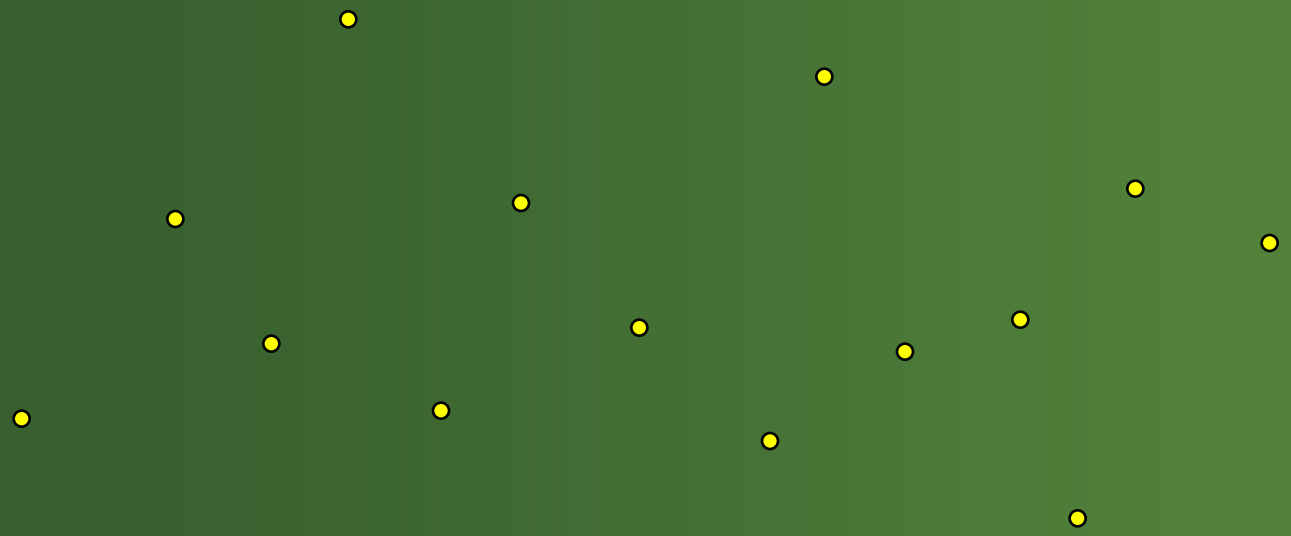
กำหนด : จุดในระนาบ 2-D



ต้นฉบับ : webspace.ship.edu/dahast/csc310/week5/closest-pair.ppt

Closest-Pair Algorithm

Step 1: เรียงลำดับจุดใน 1 D เช่น แกน x



ต้นฉบับ : webspace.ship.edu/dahast/csc310/week5/closest-pair.ppt

Closest-Pair Algorithm

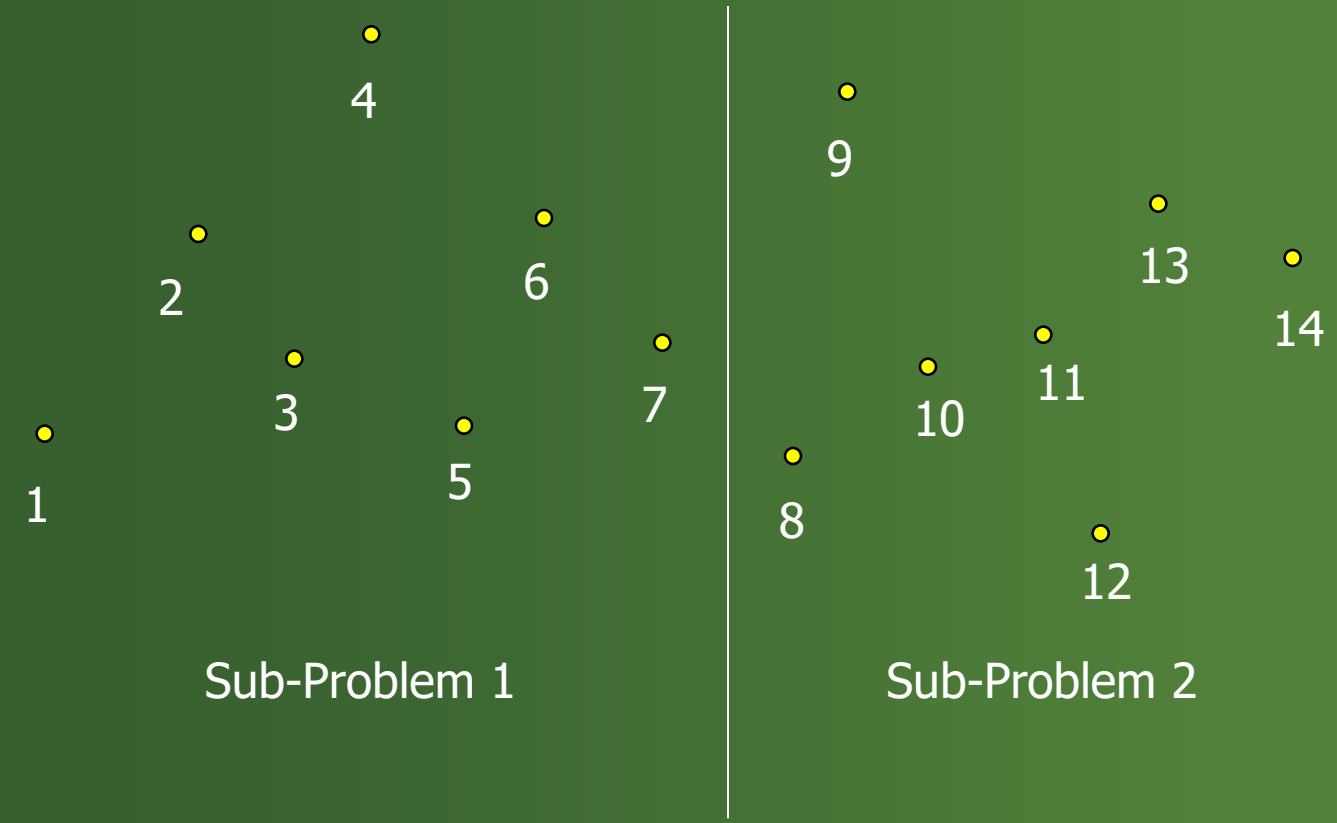
จัดเรียงตามแนว X-axis

O(n log n) เมื่อใช้ quicksort หรือ mergesort

ต้นฉบับ : webspace.ship.edu/dahast/csc310/week5/closest-pair.ppt

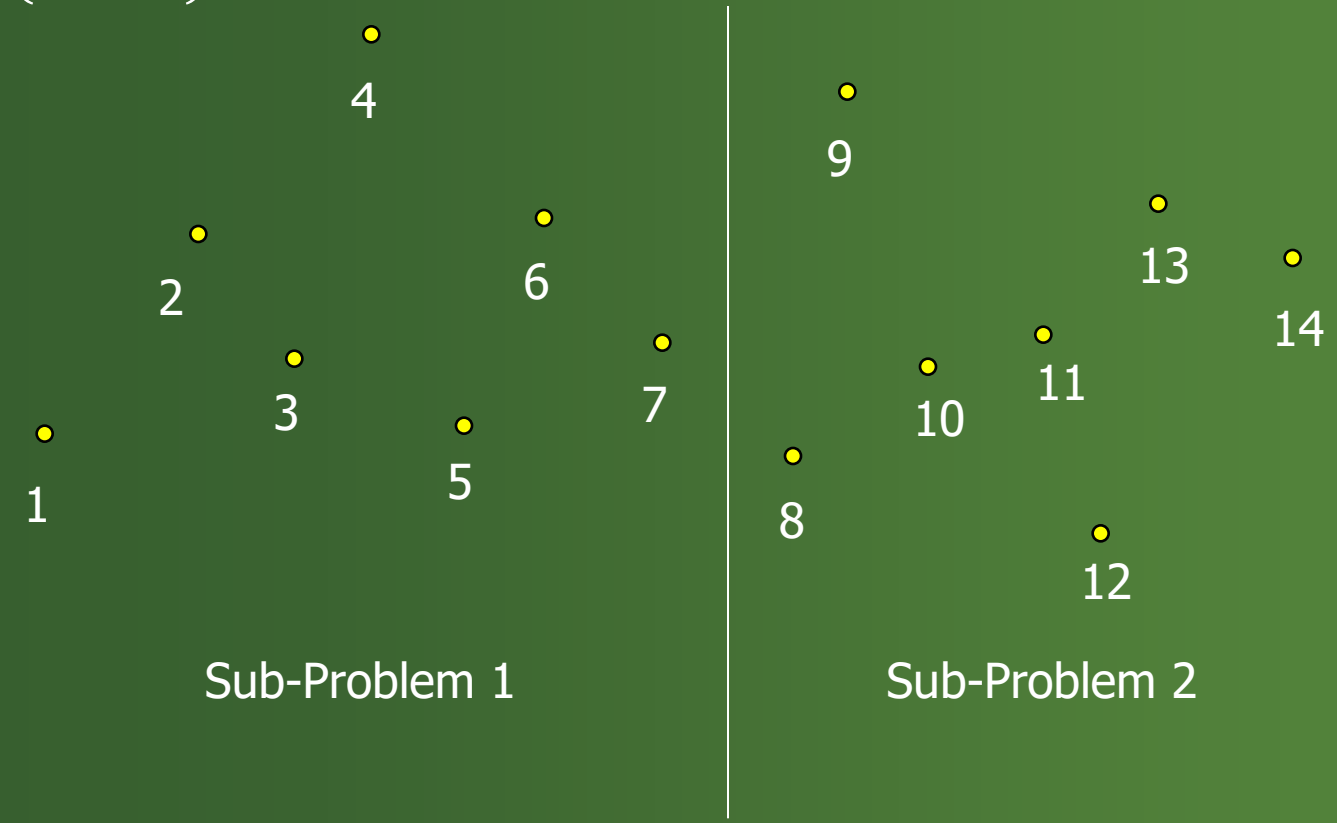
Closest-Pair Algorithm

Step 2: ลากเส้นแบ่งระหว่างกึ่งกลางของกลุ่มจุด อยู่ระหว่างจุด 7 และ 8



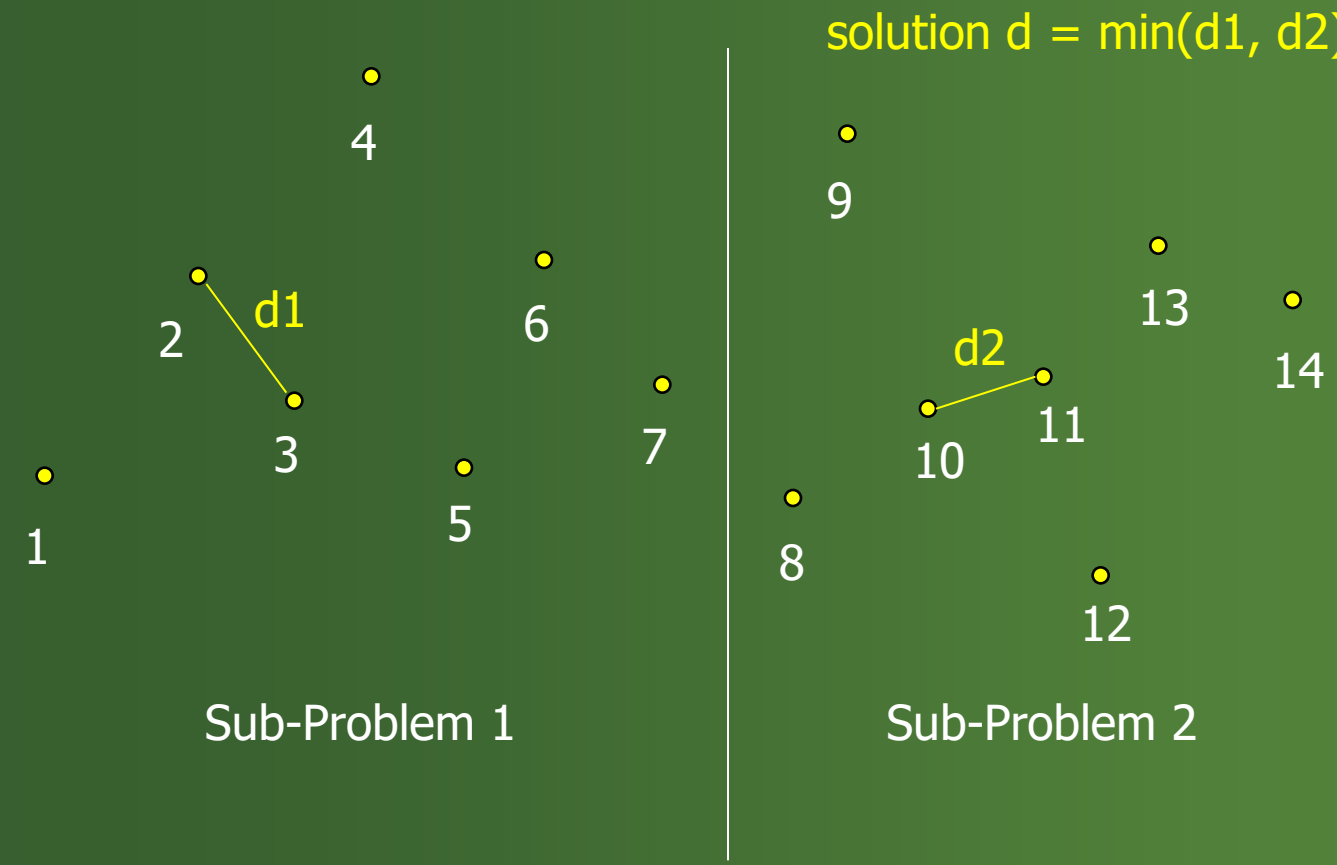
Closest-Pair Algorithm

ปกติต้องมีการเปรียบเทียบความยาวระหว่างจุด จาก 14 จุด ต้องมีการเปรียบเทียบถึง 91 ครั้ง จาก 14 เลือก 2 ได้ $\frac{14!}{2!(14-2!)}$ หรือ (n-1)n/2 = 13*14/2 = **91**



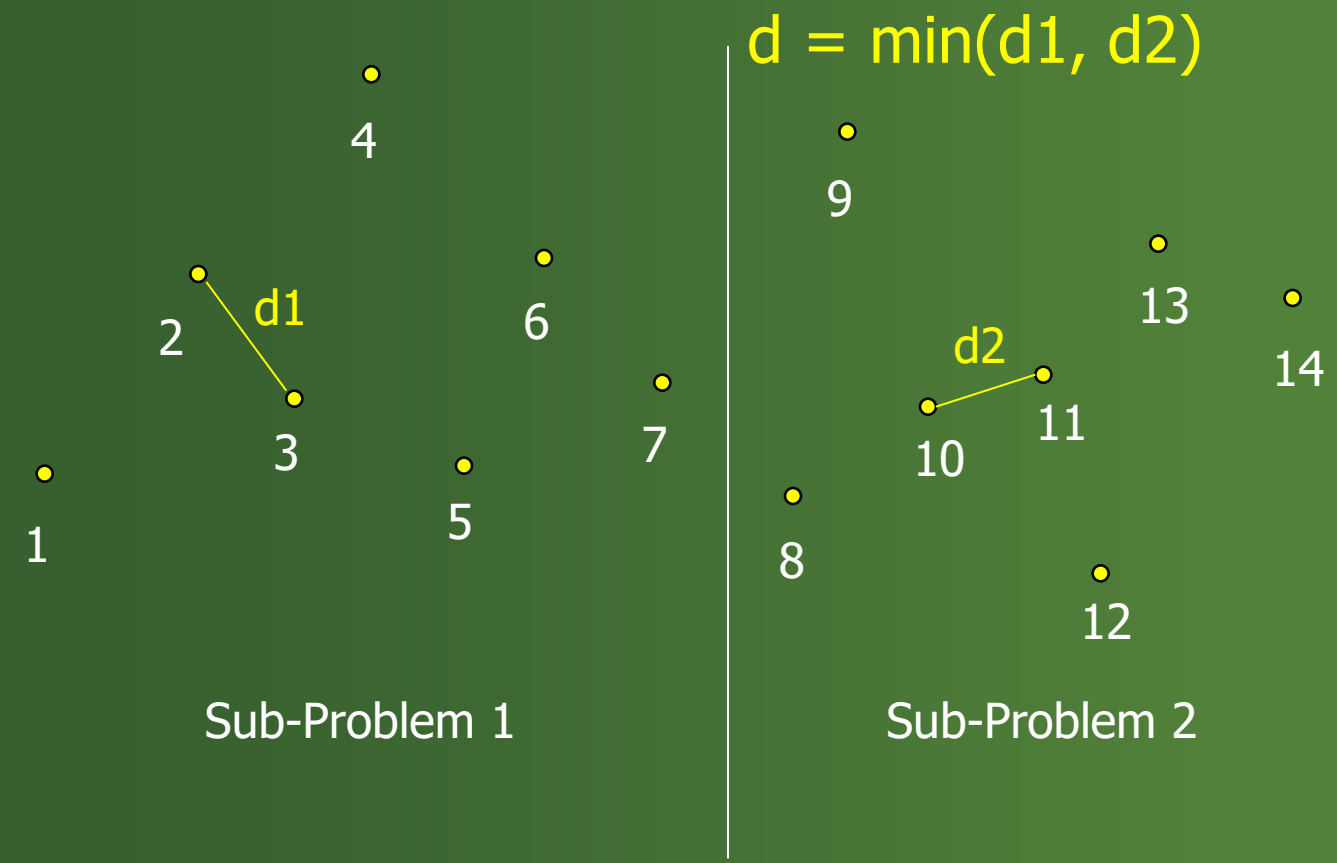
Closest-Pair Algorithm

ประโยชน์: แบ่ง sub-problems ขนาดครึ่งหนึ่ง จะทำงาน 6*7/2 ครั้งต่อข้าง รวมได้ 42 comparisons



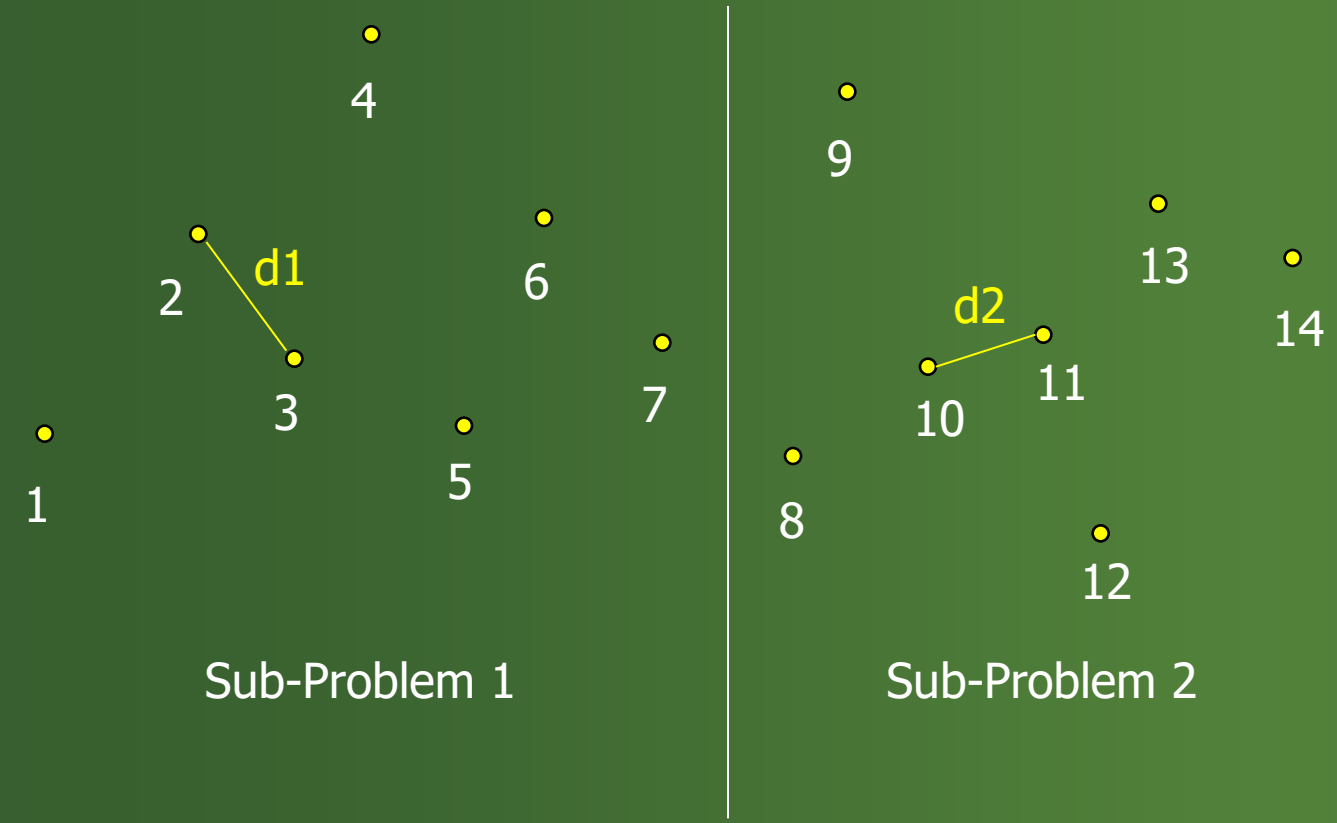
Closest-Pair Algorithm

ประโยชน์: จำนวนเปรียบเทียบลดลงเห็นได้ชัด



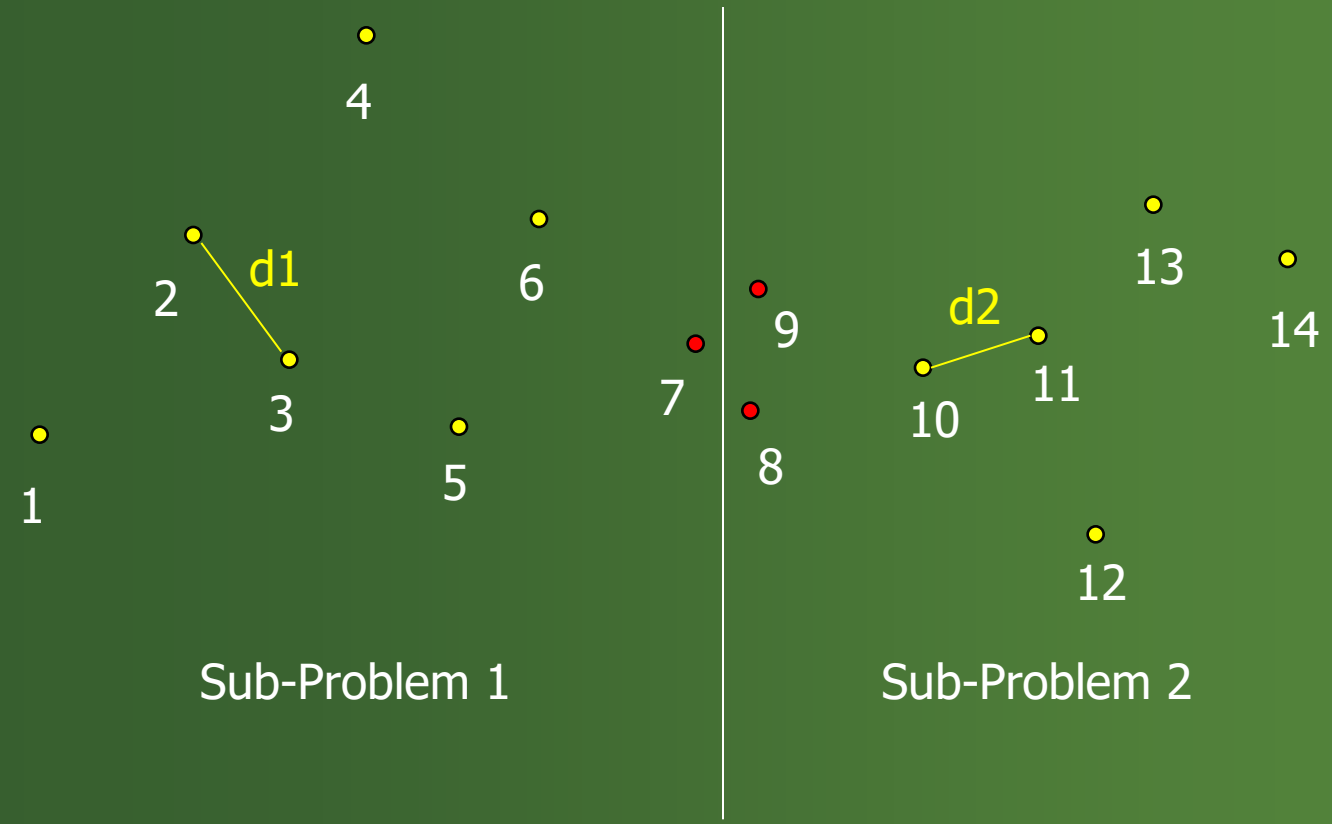
Closest-Pair Algorithm

ปัญหา: ถ้ามีระทางระหว่างจุด 2 จุดที่น้อยที่สุด ที่แต่ละจุดอยู่ different sub-problems?



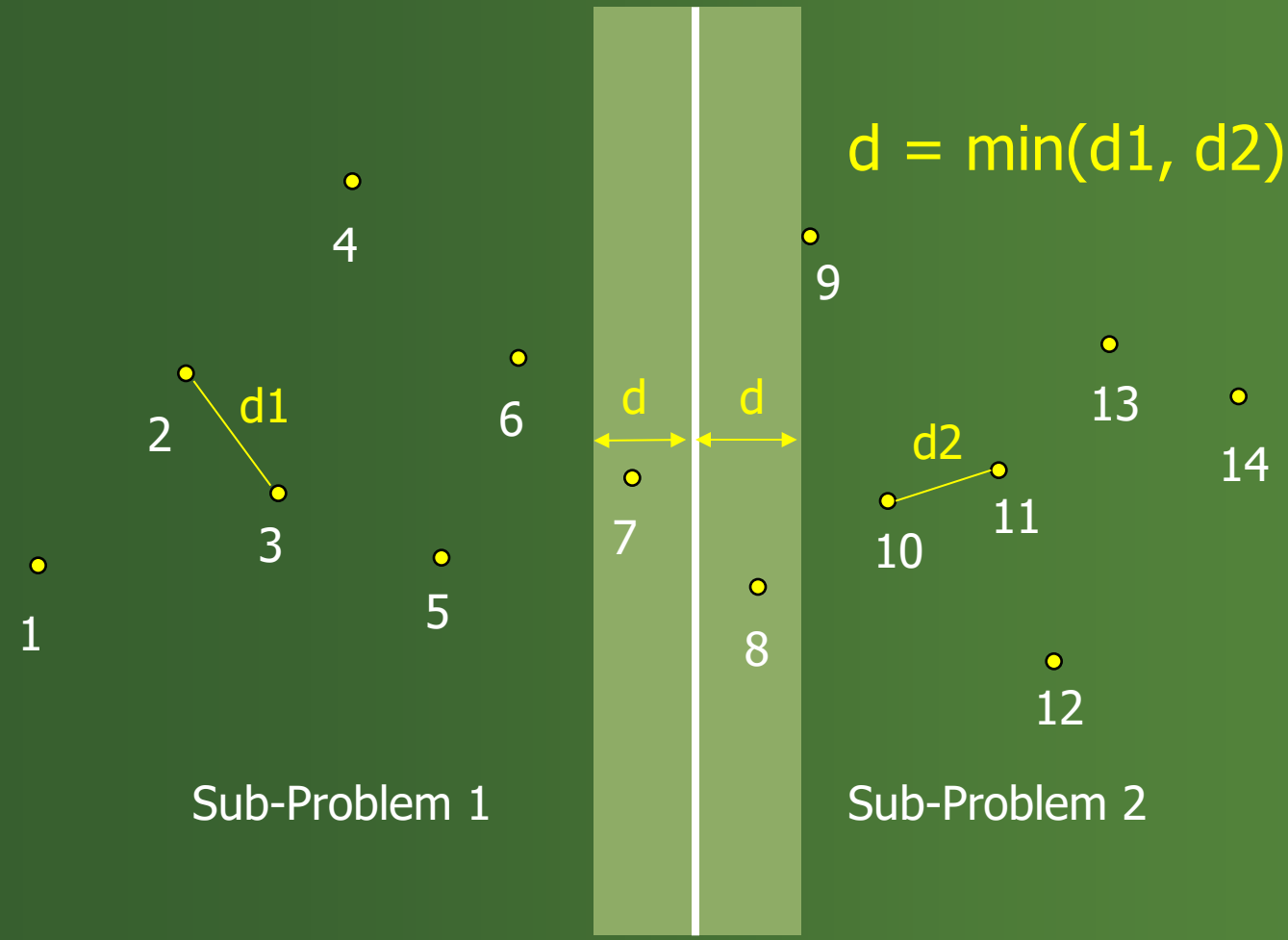
Closest-Pair Algorithm

Here is an example where we have to compare points from sub-problem 1 to the points in sub-problem 2.



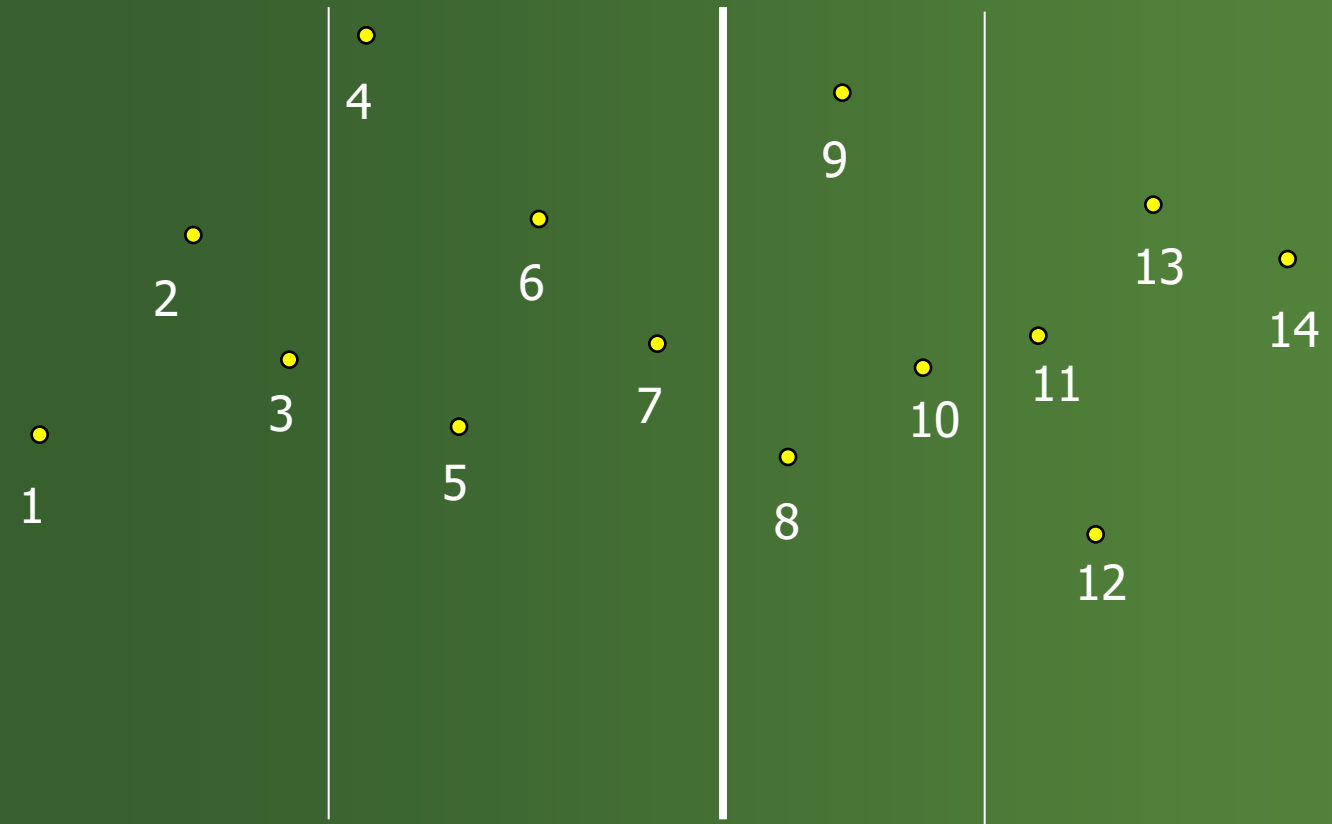
Closest-Pair Algorithm

ต้องมีการเปรียบเทียบใน “strip.”



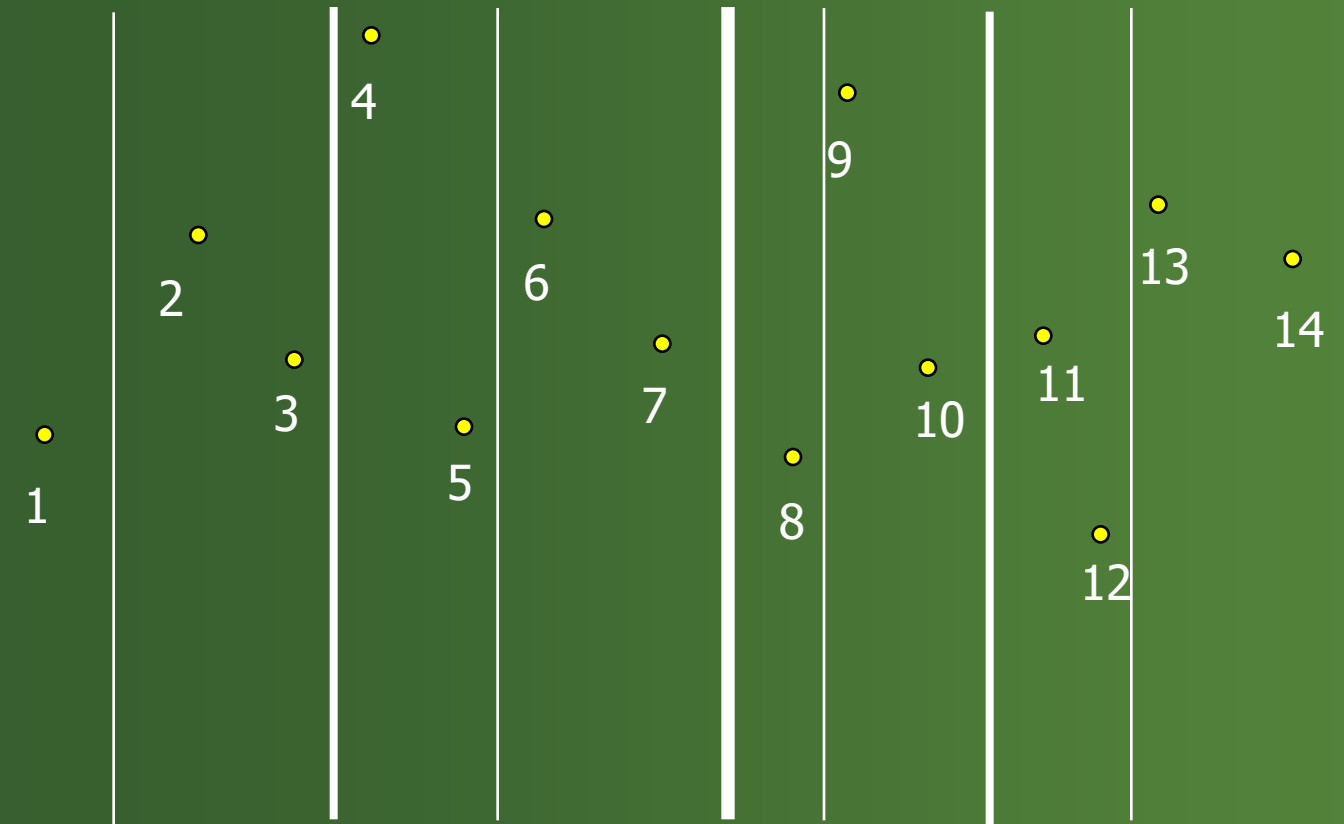
Closest-Pair Algorithm

Step 3: But, we can continue the advantage by splitting the sub-problems.



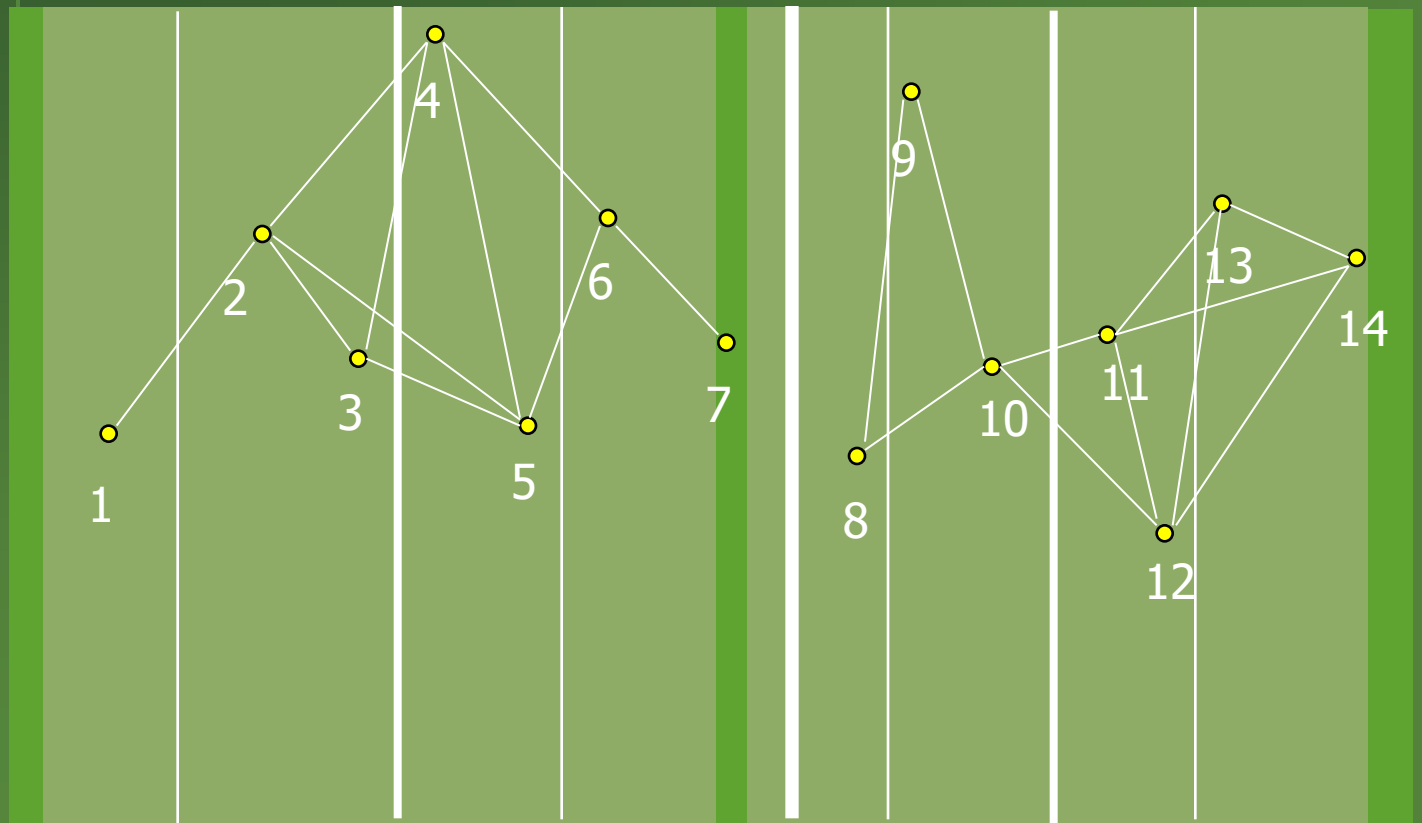
Closest-Pair Algorithm

Step 3: In fact we can continue to split until each sub-problem is trivial, i.e., takes one comparison.



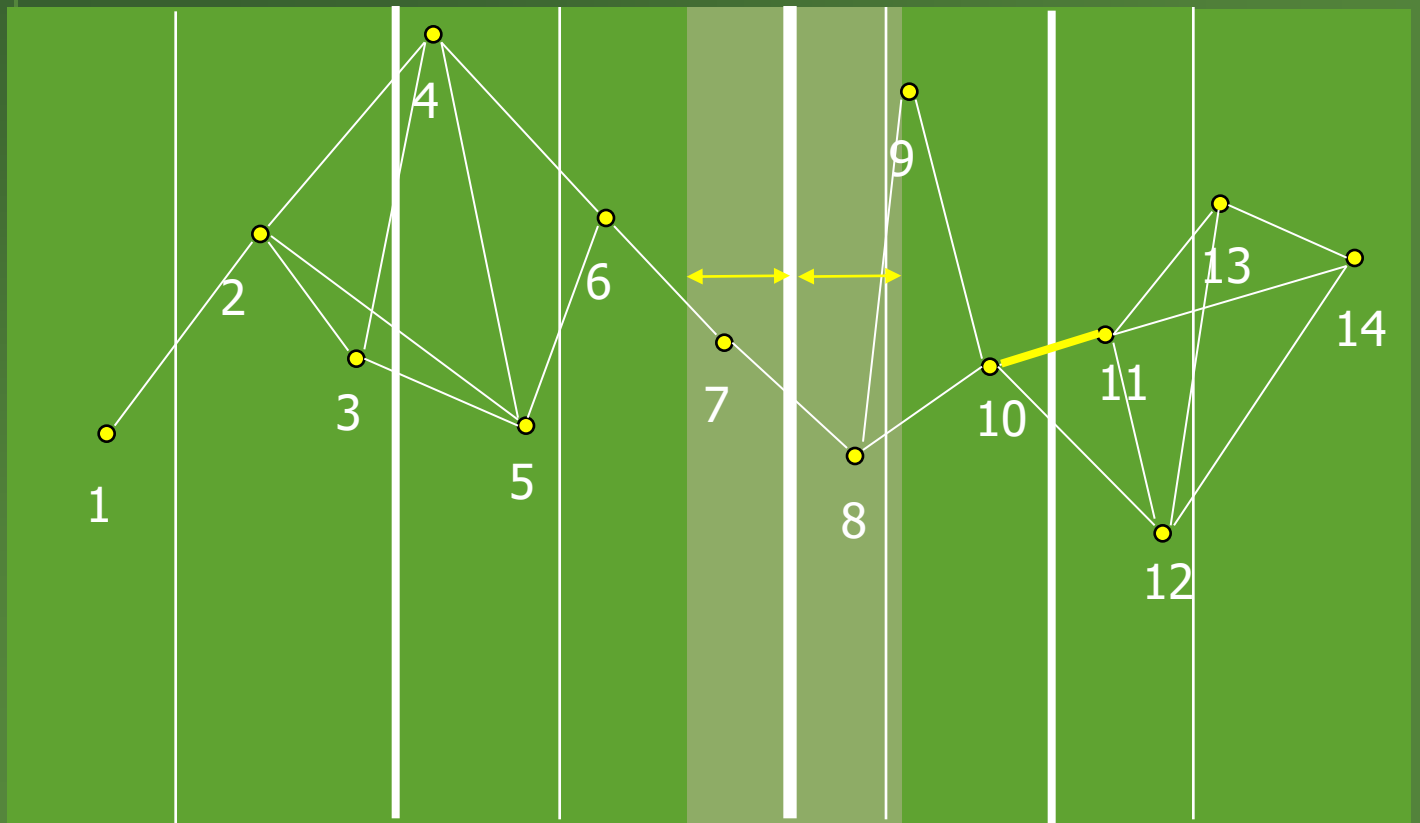
Closest-Pair Algorithm

Finally: The solution to each sub-problem is combined until the final solution is obtained



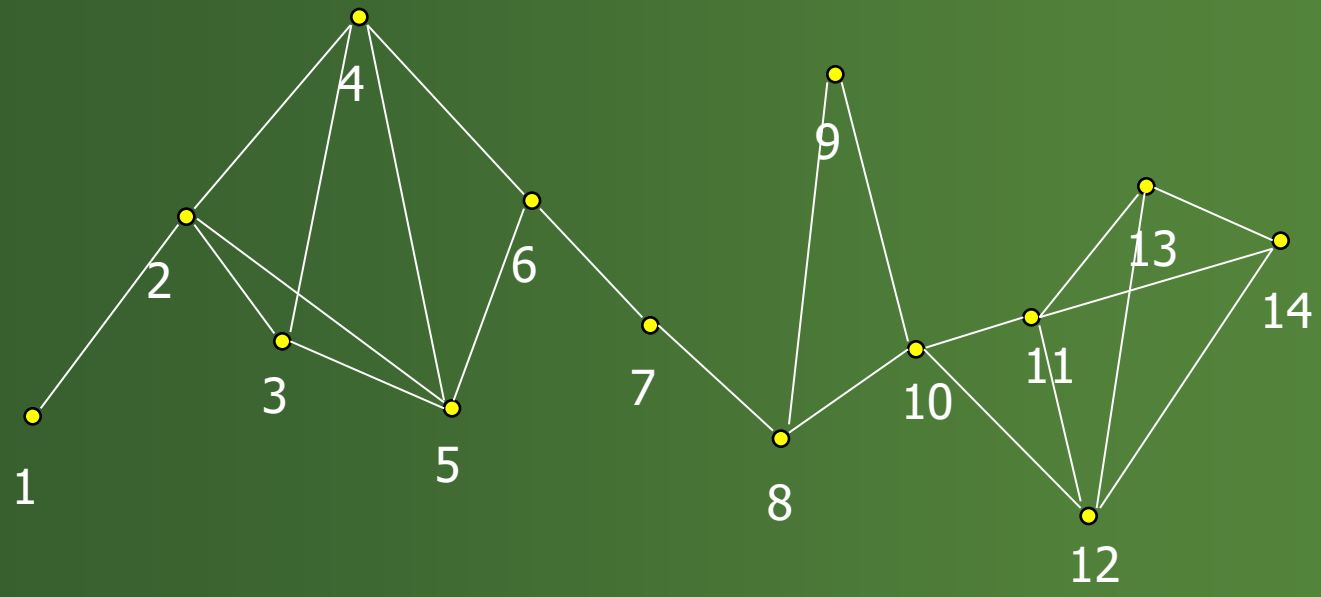
Closest-Pair Algorithm

Finally: On the last step the 'strip' will likely be very small. Thus, combining the two largest sub-problems won't require much work.



Closest-Pair Algorithm

- In this example, it takes 22 comparisons to find the closets-pair.
- The brute force algorithm would have taken 91 comparisons.
- But, the real advantage occurs when there are millions of points.



Closest-Pair Problem

ALGORITHM *EfficientClosestPair*(P, Q)

```
//Solves the closest-pair problem by divide-and-conquer
//Input: An array P of n ≥ 2 points in the Cartesian plane sorted in
//       nondecreasing order of their x coordinates and an array Q of the
//       same points sorted in nondecreasing order of the y coordinates
//Output: Euclidean distance between the closest pair of points
if n ≤ 3
    return the minimal distance found by the brute-force algorithm
else
    copy the first ⌈n/2⌉ points of P to array P_l
    copy the same ⌈n/2⌉ points from Q to array Q_l
    copy the remaining ⌊n/2⌋ points of P to array P_r
    copy the same ⌊n/2⌋ points from Q to array Q_r
    d_l ← EfficientClosestPair(P_l, Q_l)
    d_r ← EfficientClosestPair(P_r, Q_r)
    d ← min{d_l, d_r}
    m ← P[⌈n/2⌉ − 1].x
    copy all the points of Q for which |x − m| < d into array S[0..num − 1]
    dminsq ← d^2
    for i ← 0 to num − 2 do
        k ← i + 1
        while k ≤ num − 1 and (S[k].y − S[i].y)^2 < dminsq
            dminsq ← min((S[k].x − S[i].x)^2 + (S[k].y − S[i].y)^2, dminsq)
            k ← k + 1
return sqrt(dminsq)
```

Closest-Pair Problem

$$T(n) = 2T(n/2) + f(n),$$

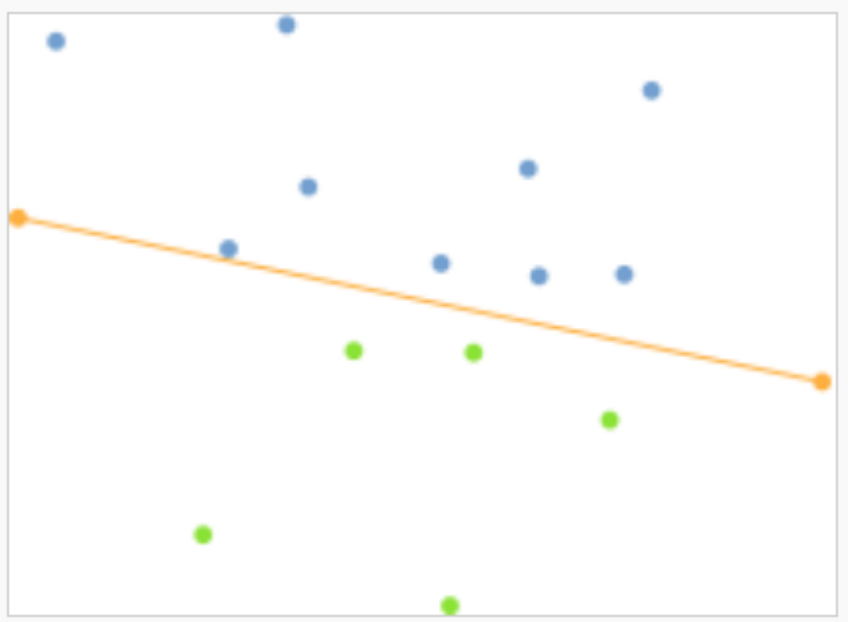
$f(n) \in \Theta(n)$.

Applying the Master Theorem (with $a = 2$, $b = 2$, and $d = 1$),

$$T(n) \in \Theta(n \log n)$$

Convex-Hull Problem

Quick hull

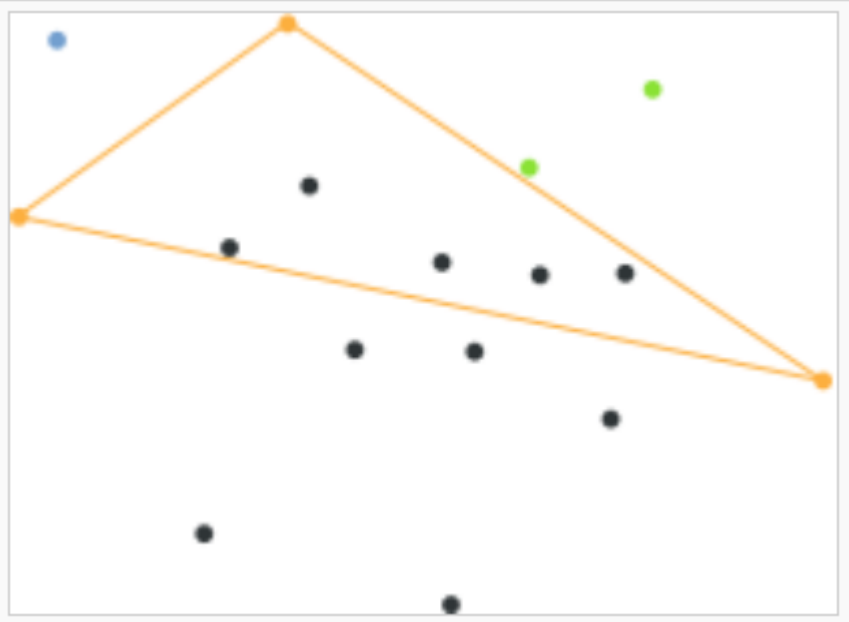


1. หา xmin และ xmax แล้วลากเส้นแบ่งเป็น 2 ส่วน

http://en.wikipedia.org/wiki/QuickHull

Convex-Hull Problem

Quick hull

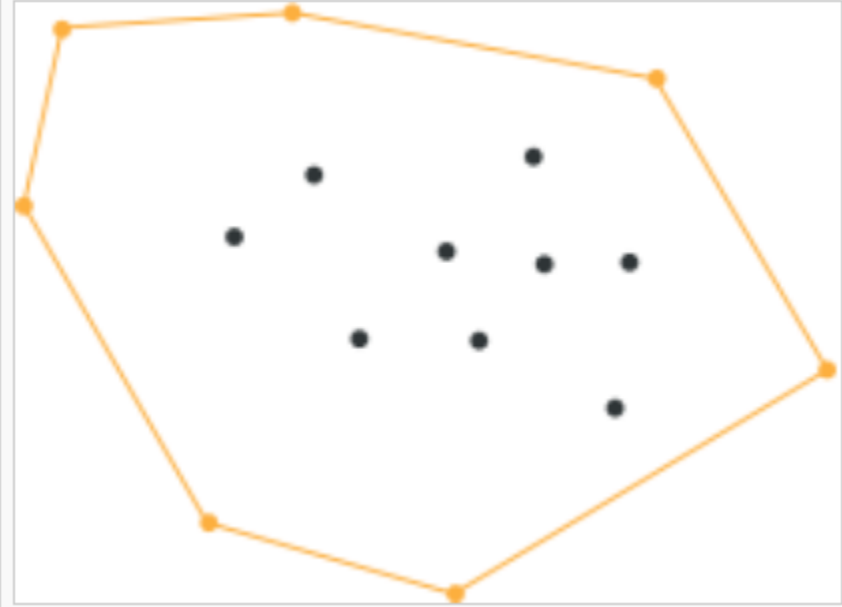


2. หาจุดที่สูงสุดของส่วนบนแล้วลากเส้นครอบจุดที่ไม่ต้องการ ทำซ้ำ

http://en.wikipedia.org/wiki/QuickHull

Convex-Hull Problem

Quick hull



3. ทำซ้ำ จนไม่มีจุดใดอยู่นอกขอบเขต

http://en.wikipedia.org/wiki/QuickHull

Convex-Hull Problem

Quick hull

ประสิทธิภาพ ในกรณีแย่ที่สุด

$$\Theta(n^2)$$

ประสิทธิภาพ ในกรณีทั่วไป

$$\Theta(n \log n)$$

http://en.wikipedia.org/wiki/QuickHull

อ้างอิงและต้นฉบับ

[1] Introduction to the design & analysis of algorithm, 3rd, Anany Levitin